Nearest-neighbor Image Resizing using Python Multiprocessing

by Marcos Fernández Hernández
for Parallel Programming for Machine Learning 2025-2026.

1. <u>Abstract</u>

• Brief problem description

This project focuses on implementing a **simple image resizer** using the **nearest-neighbor algorithm**. The goal is to compare a sequential version with a parallel implementation based on Python's **multiprocessing** module. The task consists of scaling an RGB image to a new resolution, which is a very common operation in computer vision pipelines.

• Approach summary

The sequential version goes over all output pixels and calculates the corresponding coordinates in the input image. To parallelize this, I divided the output image into **independent horizontal chunks**, so each process computes a different set of rows. Since every row can be computed **independently**, no synchronization or shared writes are needed. Once all processes are finished, their partial results are combined into the final image.

• Key results (best speedup achieved)

The parallel version improves more for larger image sizes. The best speedup obtained was approximately **4.12×** when resizing an image to **3000×3000 pixels** using **8 processes**.

2. <u>Introduction</u>

• Problem description and motivation

In this project I implemented a simple image-resizing algorithm and studied how it behaves when executed in **parallel**. Image resizing is a very common operation in computer vision and machine-learning pipelines, and it basically consists of creating a bigger or smaller version of an input image.

• Computational challenges

Although the algorithm itself is not complicated, the amount of work can be quite high for large output sizes. A resize from 3000×3000 pixels already means 9 million output pixels, and each one of them needs to be computed from its corresponding position in the input image. In Python this **can become slow**, especially because it does not run native machine code like C or C++.

Another challenge is that Python threads cannot truly run in parallel because of the **Global Interpreter Lock** (GIL). For this reason, if we want to use all CPU cores, we need to work with **multiprocessing**, which creates **multiple processes** instead of threads. This introduces some overhead, so it is not obvious beforehand whether the parallel version will actually be faster, or by how much.

• Objectives and scope

The goal of the project is to:

1. Implement a correct **sequential version** of a **nearest-neighbor image resizer**
2. Design a **parallelization strategy** using Python's **multiprocessing** module
3. Measure how **performance changes** with different numbers of processes and different output image sizes (multiple runs, warm-ups, statistics, etc.).
4. Understand how the **overhead** of multiprocessing affects small tasks
5. Analyze the results and discuss why the **speedups** behave the way they do.

The project focuses on **nearest-neighbor resizing**, since it is the simplest interpolation method and it is enough for studying parallel performance. The aim is basically to see how much **speedup** we can get when the workload grows, and how far the parallel version can go depending on the image size.

3. <u>Algorithm Description</u>

• Sequential algorithm explanation

The resizing method used in this project is **nearest-neighbor interpolation**. It is probably the simplest way to scale an image: for each pixel in the output image, we find the position in the original image that corresponds to it and copy that value directly.

If the input image has size H_in × W_in, and we want an output of H_out × W_out, then each output coordinate (yout, xout) corresponds to an input coordinate:

$$y_{in} = \left[\frac{y_{out} \cdot H_{in}}{H_{out}} \right] \qquad x_{in} = \left[\frac{x_{out} \cdot W_{in}}{W_{out}} \right]$$

The sequential version simply loops through all rows and columns of the output image and applies the formula above.

• Identification of parallelizable sections

The nice part of this algorithm is that every output pixel can be **computed independently**. Computing one pixel does not affect any other pixel, and there is no shared state that needs protection. This makes the algorithm very easy to parallelize: the output image can simply be **divided into different groups of rows,** and each process computes its own part without interfering with the others. Once all processes finish, their results are combined to form the final image.

4. <u>Parallel Implementation</u>

• Parallelization strategy

The idea of the parallel implementation is to **divide the output image** into several **blocks of rows** and let each process compute one block independently. Since computing one pixel does not depend on any other pixel, the work is naturally easy to split. Each process runs the

same function, which applies the **nearest-neighbor formula** to its assigned rows. At the end, the main program receives the partial results and places them in the correct positions of the final image.

• Process-based parallel execution used

Instead of OpenMP, this project uses **Python's multiprocessing module** to run several processes in parallel. Unlike Python threads, processes are not affected by the Global Interpreter Lock (GIL), so each one can execute at the same time on a different CPU core.

To distribute the work, I used a **multiprocessing Pool**, which is the closest equivalent to OpenMP's parallel for constructs. The idea is the same: the total work (in this case, the rows of the output image) is divided into chunks, and each chunk is assigned to one worker. The function pool.map() automatically sends each task to a process and waits until all processes finish.

• Synchronization mechanisms

No special synchronization mechanisms were needed. Each process writes only to its own **NumPy array** (representing a block of rows), so there are no shared writes and therefore **no risk of race conditions**. The only implicit synchronization is when the main process waits for all workers to finish before assembling the final image.

• Data structures and memory layout

The images are stored as **NumPy arrays** with shape (height, width, 3). This makes it easy to access RGB values and to create blocks of rows without copying unnecessary data. Each process returns its computed block as another NumPy array, which is then inserted into the final output image.

• Challenges and solutions

The first challenge was understanding how to **pass large arrays** to different processes efficiently. Since Python processes do not share memory by default, I worried that copying the image for every process would be slow. In practice, the overhead was acceptable because the image is only read, not modified. Another small challenge was finding a simple way to divide the rows evenly; this was solved by computing the block size with integer division and handling the last block separately if needed.

5. <u>Experimental Setup</u>

• Hardware specifications

All experiments were run on my personal laptop. The relevant specs are:

CPU: AMD Ryzen 7 6800H

Cores / Threads: 8 cores / 16 threads

RAM: 16 GB

Cache: 512 KB L1, 4 MB L2, 16 MB L3

OS: Windows 11 (64-bit)

• Software environment

The project was implemented in Python 3.11, using the following libraries:
NumPy for array manipulation, **Pillow** (PIL) for loading and saving images, and **multiprocessing** for parallel execution. The operating system was Windows 11, and all experiments were run from the command line to avoid external interference. No compiler flags are involved because the code is entirely in Python.

• Dataset used

The input image is a standard PNG picture stored locally.
Its exact content does not influence the performance, since the resizing algorithm simply copies pixels based on computed coordinates. The important factor is the output size, which determines the amount of computation required.
For evaluation, I resized the same input image to three different output resolutions:
1000 × 1000, 2000 × 2000 and 3000 × 3000.

• Parameters tested

The main parameter tested was the **number of processes**:
1 process (baseline parallel version, comparable to sequential), 2, 4 and 8 processes.

```
# You can change it as you wish (1, 2, 4 and 8 were studied)
n_procs = 8
```

These values allow us to study strong scaling and compare performance across different levels of parallelism.

• Measurement methodology

Before taking any measurements, I ran the computation **twice as warm-up** to avoid cold-start effects. Then, for each number of processes I ran the program multiple times (5-10) and record the resulting times (including sequential and parallel time, and speedup). And this for each output size configuration (total of 3 sizes studied). From these runs I computed the required statistics (mean, standard deviation, minimum and maximum). All measurements were taken with the system in idle conditions to minimize variability.

6. Results and Analysis

• Correctness validation

To make sure that the parallel implementation was correct, I compared the output image produced by the sequential version with the one produced using multiple processes. I used NumPy to check that both arrays were equal. In all experiments, the results matched, which confirms that dividing the image into row blocks does not affect correctness.

```
# Validation: the parallel output should be identical to the sequential one
same_output = np.array_equal(resized_seq, resized_par)
print("Outputs equal? ", same_output)
```

This should appear in the output screen:

```
Outputs equal?  True
```

• Performance results:

For each **number of processes** configuration (1, 2, 4, 8), after 2 warm-up executions, I ran the program multiple times and computed the **mean**, **standard deviation**, **minimum, maximum** and the average **Speedup** (Sequential/Parallelized time). I did this for the three output image sizes tested (1000×1000, 2000×2000, and 3000×3000).

1000x1000

Processes	Mean (ms)	Std Dev (ms)	Min	Max	Speedup
1	678.5	15.6	664.1	704.2	0.65×
2	469.1	8.8	456.1	479.2	0.93×
4	378.7	5.9	370.7	385.1	1.17×
8	405.2	14.9	391.6	431.7	1.09×

2000x2000

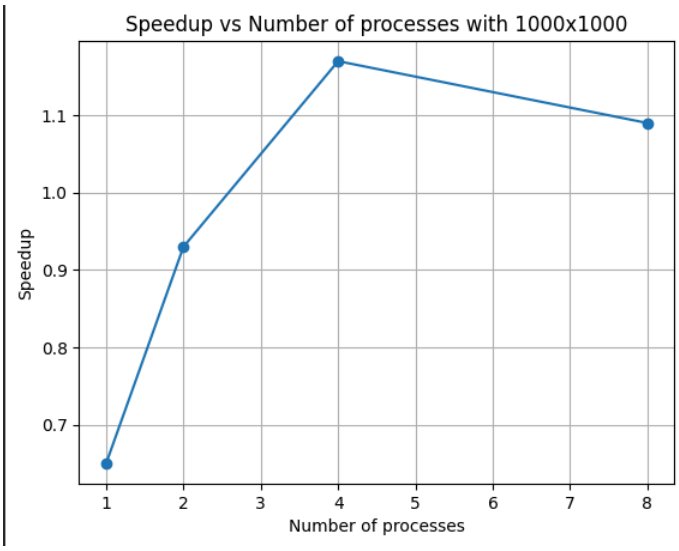
Processes	Mean (ms)	Std Dev (ms)	Min	Max	Speedup
1	1986.3	5.2	1979.5	1992.8	0.89×
2	1169.0	33.3	1144.5	1235.0	1.54×
4	729.0	2.7	725.3	732.5	2.48×
8	597.0	21.2	579.1	637.7	2.92×

3000x3000

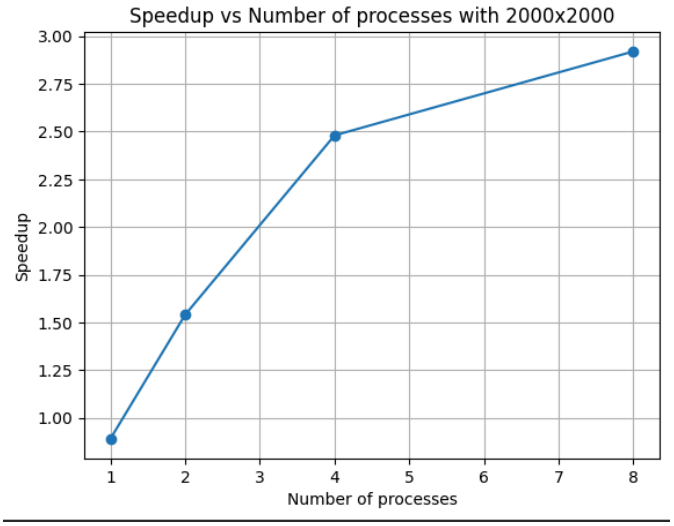
Processes	Mean (ms)	Std Dev (ms)	Min	Max	Speedup
1	4195.2	33.1	4145.9	4235.3	0.96×
2	2274.1	8.6	2263.9	2290.1	1.73×
4	1323.2	13.3	1308.3	1340.4	3.00×
8	945.7	36.6	914.6	1015.7	4.12×

• Speedup curves (speedup vs processes)

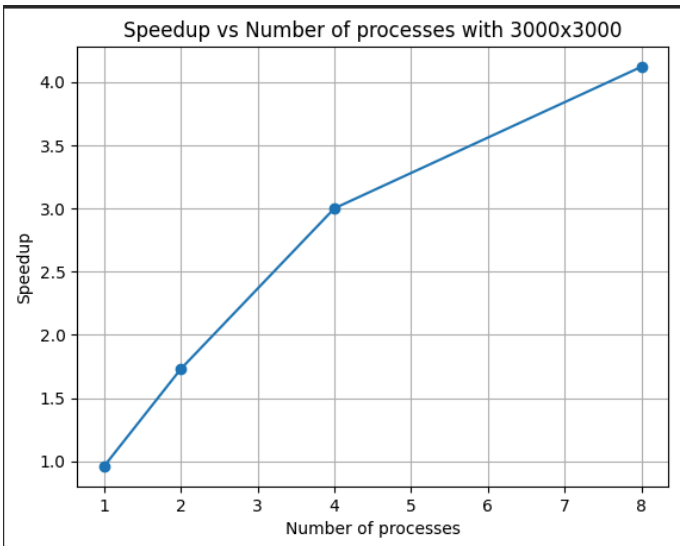
This is the representation of the **Speedup** compared to the **Number of processes**:



For the smallest size (1000×1000), the curve **rises slightly** when using 4 processes but then decreases again with 8 processes. In this case, the computation is too small to compensate for the **overhead** of creating multiple processes, so the speedup never becomes significant.



For 2000×2000, the speedup increases more consistently. The curve grows up to around **2.5× with 4 processes** and continues rising to almost **3× with 8 processes**. The workload is already large enough for multiprocessing to be beneficial, and the overhead becomes less dominant.



For the largest size (3000×3000), the curve shows the **best scaling**. The speedup increases almost linearly up to 4 processes and still improves noticeably at 8 processes, **reaching about 4×**. This happens because the amount of computation per process is much higher, so the fixed overhead of multiprocessing becomes relatively small.

In general, the curves confirm that the program **scales better** as the **output size grows**. When the image is small, the **overhead limits performance**. When the image is large, the parallel version can take advantage of multiple CPU cores much more effectively.

• Scalability analysis (strong/weak scaling)
The resize algorithm exhibits clear **strong-scaling** behavior for larger image sizes. (Weak scaling is not relevant here because the input size remains fixed.)
As the number of processes increases:
1000×1000: almost no improvement (task too small).
2000×2000: moderate scaling (up to 3×).
3000×3000: **best scaling (up to 4×)**.
This matches the expectation that the algorithm needs a large number of output pixels to take advantage of multiprocessing.

• Effect of different parameters (scheduling, chunk size, etc.)
In this project the main parameter that affects performance is the **number of processes**.
The **chunk size** for each process is **determined automatically** by dividing the output height by the number of processes. Since the work is evenly distributed, there was no need to modify the chunk size manually.
No scheduling strategies were used, since **pool.map()** handles task distribution internally.

• Comparison of variants: different parallelization strategies
Only one parallelization strategy was implemented: **splitting** the output image **by contiguous rows**. Other possible approaches (such as dividing the image into tiles or columns) were not tested because the row-based approach was the simplest to implement and already independent across processes.

• Bottleneck analysis
The main factors limiting scalability are:
Overhead of process creation: Multiprocessing is **expensive** for small tasks.
Data transfer to processes: Each worker receives a copy of the input image.
Amdahl's Law: Some parts of the program (setup, result assembly) are sequential.
For large images (3000×3000), these overheads become small relative to the computation, which explains why speedups improve significantly.

7. <u>Discussion</u>

• Interpretation of results
The results show that the performance of the parallel version depends strongly on the **size of the output image**. For small images (1000×1000), parallelization provides almost no benefit,

even in some cases it has a lower than 1 speedup, it is even slower than the sequential time. The **overhead** of creating processes and sending data to them is larger than the actual computation they perform.
For medium and large images (2000×2000 and 3000×3000), the speedup increases much more clearly. The computation becomes heavy enough for multiprocessing to **hide its overhead**, and the program is able to take advantage of multiple CPU cores. The **best speedup** obtained was around **4×** for the **largest size with 8 processes.**

• Comparison with theoretical expectations (Amdahl's Law, Gustafson's Law)
The behavior observed agrees with **Amdahl's Law**, which states that the maximum speedup is **limited** by the **sequential** part of the program. In this case, **image loading**, **task preparation** and **final assembly** are sequential steps, so perfect linear scaling is not possible.
For large images, the parallel portion dominates the total execution time, which explains why the speedup improves as the problem size grows. This is also consistent with **Gustafson's Law**, which suggests that **scalability improves** when the **workload** per processor **increases**.

• Overhead analysis
The main source of overhead comes from Python's multiprocessing:
Creating multiple processes is **expensive**.
The input image must be **copied to each** process.
Communication between processes **adds latency**.
These overheads explain why the speedup for the 1000×1000 image is low and why speedup improves progressively as the output image size increases.

• Limitations and constraints
The main limitations of this implementation were:
The Global Interpreter Lock **prevents** the **use of threads**, so only processes can be used. I couldn't parallelize as last time with threads and OpenMP.
Processes **do not share memory**, which increases **data copying cost**.
Memory bandwidth may limit performance when many processes read from the **same input array**.
The program performs many **Python-level operations**, which are **slower** compared to vectorized or compiled implementations.
And the already mentioned multiple times limitation, running with too many processes can reduce performance instead of improving it, especially for **small images**.

• Lessons learned
I learned that not all problems **benefit equally** from **parallelization**. Even if the algorithm is easy to split, the **overhead** of multiprocessing **can dominate** the total time when the workload is small. I also learned that **problem size** has a direct impact on **scalability**.

• Conclusions

In conclusion, **nearest-neighbor resizing** is **simple** to parallelize because each pixel can be computed **independently**. The multiprocessing version is **effective** for **large** images, where the computation per process is high enough to compensate for overhead. For small images, the sequential version is faster.

The results confirm the importance of understanding both the algorithm and the runtime environment when designing parallel solutions.

• Summary of achievements

Correct **sequential** and **parallel** implementations.

Successful use of **multiprocessing** to divide the work.

Full correctness **validation** between **sequential** and **parallel outputs**.

Built a measurement system with warm-up runs and multiple repetitions.

Complete **performance evaluation** across **three** image **sizes**.

Speedup up to **4×** with **8 processes** for the largest image.

• Best configuration found

The best configuration found, on average, was with **8 processes** and the **largest image size** (3000×3000), with a **mean of 945.7 ms** and an **average speedup of 4.12×** but the one that got the best **singular** result was using also **8 processes and the largest image size**, with **925 ms of execution time** and a **speedup of 4.29x**.

• Future work suggestions

In the future, the project could be extended by testing the program on different machines to see how the results vary depending on the hardware.

8. References

• Academic papers, documentation, dataset sources

My main documentation was the one on the slides on this course and the official Python and NumPy documentation. I also used a python program to create the graphics to show the curves, and finally, I made some references to the code, to relate directly to the code and the explanation.

All the outside references are shown in the commentaries of the document.